

Tutorial: **git pull** no GitHub via Terminal (HTTPS)

Guia validado para configurar autenticação persistente ao puxar (e empurrar) código do GitHub pela linha de comando, em maio de 2026.

Contexto importante

Desde **13 de agosto de 2021**, o GitHub não aceita mais a senha da conta para operações Git via HTTPS. As opções válidas hoje são:

1. **HTTPS + Personal Access Token (PAT)** armazenado por um credential helper.
2. **HTTPS + GitHub CLI (gh)** — fluxo OAuth pelo navegador, sem PAT manual.
3. **SSH** — chave pública/privada (alternativa fora do escopo deste tutorial).

Os caminhos 1 e 2 abaixo cobrem as duas primeiras opções.

Caminho A — HTTPS com Personal Access Token

Passo 1. Mudar o remote de SSH para HTTPS (se aplicável)

Se você clonou o repositório com SSH, troque a URL do remote.

```
# Entre na pasta do projeto
```

```
cd /caminho/para/seu/projeto
```

```
# Veja o remote atual
```

```
git remote -v
```

Se aparecer algo como `[email protected]:OWNER/REPOSITORY.git`, troque para HTTPS:

```
git remote set-url origin https://github.com/OWNER/REPOSITORY.git
```

Substitua **OWNER** pelo dono do repositório e **REPOSITORY** pelo nome do repo. Confirme:

```
git remote -v
```

```
# origin https://github.com/OWNER/REPOSITORY.git (fetch)
```

```
# origin https://github.com/OWNER/REPOSITORY.git (push)
```

Passo 2. Gerar um Personal Access Token (PAT)

No navegador:

1. Acesse **GitHub** → **Settings** → **Developer settings** → **Personal access tokens**.
2. Escolha o tipo de token:
 - **Fine-grained tokens** (recomendado pelo GitHub): permite restringir a repositórios específicos e definir permissões granulares. Exige data de expiração.
 - **Tokens (classic)**: compatibilidade ampla com ferramentas antigas; escopo **repo** cobre pull/push em repos públicos e privados.
3. Defina um nome descritivo, uma expiração (evite "no expiration"), e marque os escopos necessários.
4. Clique em **Generate token** e **copie o token imediatamente** — ele só é exibido uma vez. Trate-o como senha.

Formatos válidos: **ghp_...** (classic) e **github_pat_...** (fine-grained).

Passo 3. Configurar um credential helper

Escolha um helper de acordo com o sistema operacional. **Evite store puro fora de máquinas pessoais isoladas**, pois grava o token em texto plano em **~/.git-credentials**.

macOS (recomendado — Keychain do sistema):

```
git config --global credential.helper osxkeychain
```

Windows (recomendado — Git Credential Manager, já incluso no Git for Windows):

```
git config --global credential.helper manager
```

Linux com ambiente gráfico (recomendado — libsecret/gnome-keyring):

Em distros baseadas em Debian/Ubuntu, pode ser necessário instalar e compilar:

```
sudo apt install libsecret-1-0 libsecret-1-dev
```

```
sudo make --directory=/usr/share/doc/git/contrib/credential/libsecret
```

```
git config --global credential.helper  
/usr/share/doc/git/contrib/credential/libsecret/git-credential-libsecret
```

Cache em memória (sem persistência em disco — bom para máquinas compartilhadas):

```
git config --global credential.helper 'cache --timeout=3600'
```

Mantém as credenciais em RAM por 1 hora

Plain text (só use em máquina pessoal de uso exclusivo):

```
git config --global credential.helper store
```

Grava em ~/.git-credentials em texto plano — qualquer um com acesso ao arquivo lê o token

Passo 4. Executar `git pull` e autenticar

```
git pull
```

Quando o terminal pedir credenciais:

- **Username:** seu usuário do GitHub.
- **Password:** cole o **PAT** (não a senha da conta).

A partir daí, o helper guarda a credencial e os próximos `git pull` / `git push` rodam sem prompt, até o token expirar.

Caminho B — GitHub CLI (`gh`)

Abordagem mais moderna: o `gh` faz login OAuth pelo navegador e configura automaticamente o Git para usá-lo como credential helper. Não precisa criar PAT manualmente.

Passo 1. Instalar o GitHub CLI

- **macOS:** `brew install gh`
- **Linux (Debian/Ubuntu):** instruções em <https://cli.github.com> (pacote `gh`)
- **Windows:** `winget install --id GitHub.cli` ou `choco install gh`

Passo 2. Fazer login

```
gh auth login
```

Responda às perguntas:

1. **What account do you want to log into?** → `GitHub.com`
2. **What is your preferred protocol for Git operations on this host?** → `HTTPS`
3. **Authenticate Git with your GitHub credentials?** → `Y` (essencial — é o que configura o `gh` como credential helper do Git)
4. **How would you like to authenticate?** → `Login with a web browser`

Copie o código de uso único exibido, abra a URL indicada no navegador, cole o código e autorize.

Passo 3. Confirmar e usar

```
gh auth status
```

Deve mostrar "Logged in to github.com" e "git operations protocol: https"

git pull

Roda direto, sem prompts

Quando o token OAuth expirar, basta rodar `gh auth login` novamente.

Diagnóstico rápido

Sintoma	Causa provável	Solução
<code>Support for password authentication was removed on August 13, 2021</code>	Você digitou a senha da conta em vez de um PAT	Gere um PAT e use-o como senha
Git pede credencial em todo <code>pull</code>	Nenhum credential helper configurado	Rode <code>git config --global credential.helper ...</code> (Passo 3 do Caminho A)
<code>remote: Permission denied</code> mesmo com PAT	PAT sem o escopo <code>repo</code> , ou expirado, ou (fine-grained) sem acesso àquele repositório específico	Regenere o token com os escopos/repos corretos
Quer ver qual helper está ativo	—	<code>git config --global credential.helper</code>
Quer remover credenciais salvas	—	macOS: apague a entrada <code>github.com</code> no Keychain. Windows: Credential Manager → Windows Credentials. Linux/store: apague a linha em <code>~/.git-credentials</code>

Boas práticas de segurança

- Prefira **fine-grained tokens** com escopo mínimo e expiração curta (30–90 dias).

- Use o **credential helper nativo do SO** (Keychain / Credential Manager / libsecret) em vez de **store**.
- Crie **tokens separados** para máquinas diferentes — se uma vazar, você revoga só uma.
- Nunca cole o PAT em scripts versionados, mensagens de commit ou URLs públicas.
- Revise periodicamente em **Settings** → **Developer settings** → **Personal access tokens** e revogue tokens não usados.